Laboratorio 2.2: Formulario de Registro Ultra-Resiliente y Navegación Predictiva

1. Contexto y Objetivos

En el desarrollo profesional, la pérdida de datos de un usuario por un cambio de orientación o por falta de memoria del sistema es inaceptable. Este laboratorio integra el **Ciclo de Vida de Android 16**, la **Arquitectura MVVM** y la **Persistencia Local** para crear una interfaz que "sobreviva a todo".

Objetivos:

- Implementar persistencia de estado multinivel: Memoria (ViewModel), Proceso (SavedStateHandle) y Disco (DataStore).
- Configurar el sistema de **Predictive Back** (Navegación Predictiva) nativo de la API 36.
- Aplicar el patrón de diseño de **Unidirectional Data Flow (UDF)** en Jetpack Compose.

2. Requisitos Previos

- Haber completado el **Laboratorio 1** (Estandarización) y el **Laboratorio 2.1** (Ciclo de Vida básico).
- Emulador **Pixel 8 - API 36** con "Opciones de Desarrollador" activas.
- Conocimientos básicos de **Corrutinas** (Flow/Suspend functions).

3. Descripción General

El estudiante construirá un "Borrador de Perfil" con campos de Nombre, Email y un interruptor de Notificaciones. El reto consiste en asegurar que, si el usuario escribe algo y sale de la app o rota el teléfono, los datos sigan ahí exactamente donde los dejó, utilizando las APIs recomendadas por Google para 2026.

4. Paso a Paso Detallado

Paso 0: Preparación y Dependencias (Gradle)

1. **Clonación:** Copie la carpeta de su RDA1_Lab1 y renómbrela como RDA1_Lab7_ResilientForm_NombreApellido.
2. **Limpieza:** Borre las carpetas .gradle y build. Abra el proyecto en **Android Studio Ladybug**.
3. **Configuración de Gradle:** Abra el archivo app/build.gradle.kts y añada la dependencia de DataStore dentro del bloque dependencies:

```
// app/build.gradle.kts
implementation("androidx.datastore:datastore-preferences:1.1.1")
```

4. **Sincronización:** Haga clic en el **icono del Elefante (Sync Project)** y verifique que no haya errores de red.

Paso 1: Capa de Datos (DataStore para persistencia total)

Crearemos un repositorio que guarde el borrador en el disco del teléfono.

1. Cree el archivo app/src/main/java/[tu.paquete]/data/UserPreferences.kt.
2. **Agregue** el siguiente código (reemplace [tu.paquete] por el suyo):

```
private val Context.dataStore by preferencesDataStore(name = "user_prefs")

class UserPreferences(private val context: Context) {
    private val NAME_KEY = stringPreferencesKey("user_name")

    // Flujo de datos para leer el nombre
    val userName: Flow<String> = context.dataStore.data.map { it[NAME_KEY] ?: "" }

    // Función para guardar el nombre en disco
    suspend fun saveName(name: String) {
        context.dataStore.edit { it[NAME_KEY] = name }
    }
}
```

Paso 2: Capa de Lógica (ViewModel con SavedStateHandle)

El ViewModel gestionará el estado de la UI y la persistencia ante muerte del proceso.

1. Cree el archivo app/src/main/java/[tu.paquete]/ui/FormViewModel.kt.
2. **Agregue** este código:

```
class FormViewModel(
    private val stateHandle: SavedStateHandle,
    private val userPrefs: UserPreferences
) : ViewModel() {

    // Estado que sobrevive a Muerte del Proceso (Process Death)
    var email by mutableStateOf(stateHandle.get<String>("email_key") ?: "")
        private set

    fun updateEmail(newEmail: String) {
        email = newEmail
        stateHandle["email_key"] = newEmail
    }

    // Estado que sobrevive a Cierre de App (DataStore)
```

```
    val nameFromDisk = userPrefs.userName.asLiveData()

    fun saveName(newName: String) {
        viewModelScope.launch { userPrefs.saveName(newName) }
    }
}
```

Paso 3: Interfaz de Usuario y Predictive Back (API 36)

Implementaremos el formulario y el sistema de navegación moderna.

1. Abra MainActivity.kt.
2. **Reemplace** el contenido de MainActivity para incluir el BackHandler:

```
@Composable
fun ResilientFormScreen(viewModel: FormViewModel) {
    val nameDisk by viewModel.nameFromDisk.observeAsState("")
    var nameInput by remember { mutableStateOf(nameDisk) }

    // Implementación de Predictive Back (Navegación de Android 16)
    BackHandler(enabled = nameInput.isNotEmpty()) {
        // Lógica para interceptar el regreso si hay cambios sin guardar
        Log.d("NAV", "El usuario intentó retroceder con datos en el formulario")
    }

    Column(modifier = Modifier.fillMaxSize().padding(24.dp)) {
        Text("Borrador de Perfil", style = MaterialTheme.typography.headlineMedium)

        // Campo persistido en DISCO (DataStore)
        OutlinedTextField(
            value = nameInput,
            onValueChange = {
                nameInput = it
                viewModel.saveName(it)
            },
            label = { Text("Nombre (Persiste al cerrar app)") }
        )

        // Campo persistido en MEMORIA/PROCESO (SavedStateHandle)
        OutlinedTextField(
            value = viewModel.email,
            onValueChange = { viewModel.updateEmail(it) },
            label = { Text("Email (Persiste al rotar/muerte proceso)") }
        )
    }
}
```

Paso 4: Ejecución y Validación Técnica

1. **Ejecutar:** Presione el botón **Run**.
2. **Prueba A (Rotación):** Escriba en ambos campos y rote el emulador (Ctrl+F12). Los datos deben seguir ahí.
3. **Prueba B (Process Death):** Con la opción "Don't keep activities" activada (ver Lab 2.1), minimice la app y vuelva. El **Email** debe persistir.
4. **Prueba C (Cierre Total):** Cierre la aplicación por completo (Swipe up en recientes). Ábrala de nuevo. El **Nombre** debe restaurarse desde el DataStore automáticamente.

5. Resultados Esperados

- El campo de **Nombre** es "eterno": sobrevive incluso si se borra la app de la memoria RAM.
- El campo de **Email** es "resiliente": sobrevive a la rotación y a la destrucción de la actividad por el sistema.
- El sistema muestra en el **Logcat** un registro cada vez que el usuario intenta usar el gesto de "Atrás" si el formulario no está vacío (Predictive Back).

6. Desafío

Reto: Implemente un indicador visual (un pequeño icono de nube o check) que aparezca durante 1 segundo cada vez que el UserPreferences confirme que el dato se escribió en el disco exitosamente.

7. Entregables

1. Informe **PDF** con mínimo:
    - **Captura de Logcat:** Evidencia de la interceptación del **BackHandler**.
    - **URL del Repositorio:** GitHub con el README que explique la diferencia entre SavedStateHandle y DataStore.
    - **Link a un Video Demostrativo (30 seg):** Donde se vea el formulario manteniendo los datos tras cerrar la app y volver a entrar.
    - Copia del código como Anexo
    - Respuestas a preguntas
    - Demás puntos que debe contener el informe como conclusiones, recomendaciones, etc.

8. Criterios de Evaluación

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe